Although updates are rare, they can still happen, and if this cache is to be accessed from multiple threads, it will need to be appropriately protected during updates to ensure that none of the threads reading the cache see a broken data structure.

In the absence of a special-purpose data structure that exactly fits the desired usage and that's specially designed for concurrent updates and reads (such as those in chapters 6 and 7), such an update requires that the thread doing the update have exclusive access to the data structure until it has completed the operation. Once the change is complete, the data structure is again safe for multiple threads to access concurrently. Using a `std::mutex` to protect the data structure is therefore overly pessimistic, because it will eliminate the possible concurrency in reading the data structure when it isn't undergoing modification; what's needed is a different kind of mutex. This new kind of mutex is typically called a *reader-writer* mutex, because it allows for two different kinds of usage: exclusive access by a single "writer" thread or shared, concurrent access by multiple "reader" threads.

The new C++ Standard Library doesn't provide such a mutex out of the box, although one was proposed to the Standards Committee.[3] Because the proposal wasn't accepted, the examples in this section use the implementation provided by the Boost library, which is based on the proposal. As you'll see in chapter 8, the use of such a mutex isn't a panacea, and the performance is dependent on the number of processors involved and the relative workloads of the reader and updater threads. It's therefore important to profile the performance of the code on the target system to ensure that there's actually a benefit to the additional complexity.

[3] Howard E. Hinnant, "Multithreading API for C++0X—A Layered Approach," C++ Standards Committee Paper N2094, http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2006/n2094.html.

Rather than using an instance of `std::mutex` for the synchronization, you use an instance of `boost::shared_mutex`. For the update operations, `std::lock_guard<boost::shared_mutex>` and `std::unique_lock<boost::shared_mutex>` can be used for the